



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
UNIVERSITY OF SCIENCE AND TECHNOLOGY - UD



CƠ SỞ DỮ LIỆU

TS. Võ Đức Hoàng

Khoa Công nghệ Thông tin

Trường Đại học Bách khoa - Đại học Đà Nẵng

THINKING - CREATING - HUMANITY CHERISHING





ĐẠI HỌC ĐÀ NẴNG

TRƯỜNG ĐẠI HỌC BÁCH KHOA

UNIVERSITY OF SCIENCE AND TECHNOLOGY - UD



CHƯƠNG 10

SQL nâng cao

THINKING - CREATING - HUMANITY CHERISHING



NỘI DUNG

10.1. Phép toán tập hợp trong SQL

10.2. Truy vấn kết bảng: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN

10.3. Nhóm và tổng hợp dữ liệu

10.4. Truy vấn lồng nhau.

Mục tiêu của bài học

1. **Hiểu và áp dụng các kiểu JOIN** trong SQL để kết hợp dữ liệu từ nhiều bảng
2. Thực hiện truy vấn tổng hợp dữ liệu
3. **Viết được truy vấn lồng nhau (Subquery)** ở các vị trí khác nhau
4. Phân biệt được cách dùng JOIN và Subquery
5. **Kết hợp nhiều kỹ thuật SQL nâng cao** để xây dựng truy vấn phức tạp phục vụ phân tích và báo cáo dữ liệu



10.1. Phép toán tập hợp trong SQL

Phép toán tập hợp trong SQL

- SQL có cài đặt các phép toán
 - Hội (UNION)
 - Giao (INTERSECT)
 - Trừ (EXCEPT)
- Kết quả trả về là tập hợp
 - Loại bỏ các bộ trùng nhau
- Để giữ lại các bộ trùng nhau
 - UNION ALL
 - INTERSECT ALL
 - EXCEPT ALL

Phép toán tập hợp trong SQL (tt)

- Cú pháp

```
SELECT <ds cột> FROM <ds bảng> WHERE <điều kiện>  
UNION [ALL]  
SELECT <ds cột> FROM <ds bảng> WHERE <điều kiện>
```

```
SELECT <ds cột> FROM <ds bảng> WHERE <điều kiện>  
INTERSECT [ALL]  
SELECT <ds cột> FROM <ds bảng> WHERE <điều kiện>
```

```
SELECT <ds cột> FROM <ds bảng> WHERE <điều kiện>  
EXCEPT [ALL]  
SELECT <ds cột> FROM <ds bảng> WHERE <điều kiện>
```

Ví dụ 1

- Cho biết các mã đề án có
 - Nhân viên với họ là 'Nguyen' tham gia hoặc Trưởng phòng chủ trì đề án đó với họ là 'Nguyen'

```
SELECT MADA
FROM NHANVIEN, PHANCONG
WHERE MANV=MA_NVIENT AND HONV='Nguyen'
UNION
SELECT MADA
FROM NHANVIEN, PHONGBAN, DEAN
WHERE MANV=TRPHG AND MAPHG=PHONG
AND HONV='Nguyen'
```


Ví dụ 2

- Tìm nhân viên có tên cùng tên với thân nhân và cùng giới tính

```
SELECT TENNV, PHAI FROM NHANVIEN
```

```
INTERSECT
```

```
SELECT TENTN, PHAI FROM THANNHAN
```

Ví dụ 3

- Tìm những nhân viên không có thân nhân nào

```
SELECT MANV FROM NHANVIEN
```

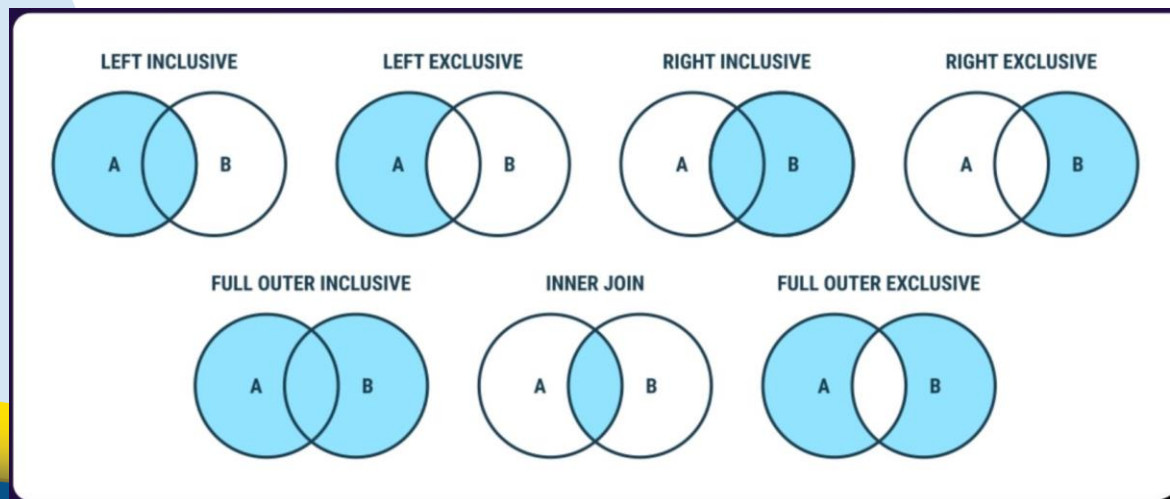
~~EXCEPT~~

```
SELECT MA_NVIAN AS MANV FROM THANNHAN
```

10.2. Truy vấn kết bảng: INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN

Tổng quan JOIN trong SQL

- **Mục đích:**
 - Kết hợp dữ liệu từ **hai hoặc nhiều bảng** dựa trên **mối quan hệ giữa các khóa chính – khóa ngoại**.
- **Ý nghĩa:**
 - Dữ liệu trong CSDL thường được **tách thành các bảng chuẩn hóa**
 - JOIN giúp **khôi phục quan hệ logic giữa các bảng** thông qua điều kiện nối



INNER JOIN

- Mục đích:
 - Lấy các dòng có giá trị trùng khớp ở cả hai bảng theo điều kiện JOIN.
- Cú pháp:

```
SELECT A.Cot1, B.Cot2
FROM BangA A
INNER JOIN BangB B ON A.Khoa = B.Khoa;
```
- Ví dụ

```
SELECT NHANVIEN.TENNV, PHONGBAN.TENPB
FROM NHANVIEN
INNER JOIN PHONGBAN ON NHANVIEN.PHG = PHONGBAN.MAPHG;
```

 - *Chỉ hiển thị nhân viên có phòng ban hợp lệ*
 - *Không lấy nhân viên chưa được phân phòng*

LEFT JOIN (LEFT OUTER JOIN)

- Mục đích:
 - Lấy toàn bộ dữ liệu từ bảng bên trái, và khớp bên phải nếu có, nếu không thì điền NULL.
- Cú pháp:

```
SELECT A.*, B.*  
FROM A LEFT JOIN B ON A.Khoa = B.Khoa;
```
- Ví dụ

```
SELECT NHANVIEN.TENNV, PHONGBAN.TENPB  
FROM NHANVIEN LEFT JOIN PHONGBAN ON NHANVIEN.PHG = PHONGBAN.MAPHG;
```

 - *Hiển thị toàn bộ nhân viên, kể cả chưa có phòng ban*
 - *Cột TENPB sẽ là NULL nếu PHG không khớp với MAPHG*

RIGHT JOIN (RIGHT OUTER JOIN)

- Mục đích:
 - Lấy **toàn bộ dữ liệu từ bảng bên phải**, kết nối bên trái nếu có, nếu không thì điền NULL.
- Cú pháp:

```
SELECT A.*, B.*  
FROM A RIGHT JOIN B ON A.Khoa = B.Khoa;
```
- Ví dụ

```
SELECT NHANVIEN.TENNV, PHONGBAN.TENPB  
FROM NHANVIEN RIGHT JOIN PHONGBAN ON NHANVIEN.PHG = PHONGBAN.MAPHG;
```

 - *Hiển thị tất cả phòng ban, kể cả phòng chưa có nhân viên*
 - *Cột TENNV sẽ là NULL nếu không có ai thuộc phòng đó*

FULL JOIN (FULL OUTER JOIN)

- Mục đích:
 - Kết hợp cả **LEFT JOIN** và **RIGHT JOIN** – lấy tất cả dòng từ cả hai bảng, nếu không khớp thì điền NULL.
- Cú pháp:

```
SELECT A.*, B.*  
FROM A FULL JOIN B ON A.Khoa = B.Khoa
```
- Ví dụ

```
SELECT NHANVIEN.TENNV, PHONGBAN.TENPB  
FROM NHANVIEN  
FULL JOIN PHONGBAN ON NHANVIEN.PHG = PHONGBAN.MAPHG;
```

 - *Lấy cả nhân viên chưa có phòng và phòng chưa có nhân viên*

So sánh các loại JOIN

- Mục đích:

Loại JOIN	Giữ tất cả từ bảng	Chỉ giữ khi khớp	Dòng có thể có NULL
INNER JOIN	✗	✓	✗
LEFT JOIN	✓ (trái)	✓	bảng phải nếu không khớp
RIGHT JOIN	✓ (phải)	✓	bảng trái nếu không khớp
FULL JOIN	✓ (cả hai)	✓	Cả hai bảng nếu không khớp

So sánh: Truy vấn FROM nhiều bảng vs JOIN

Tiêu chí	FROM nhiều bảng (liệt kê bảng)	JOIN (câu lệnh rõ ràng)
Cú pháp cơ bản	FROM A, B WHERE A.x = B.y	FROM A JOIN B ON A.x = B.y
Cách hoạt động	Dựa vào tích Đề-các (Cartesian product) rồi lọc	Dựa vào JOIN có điều kiện rõ ràng
Dễ đọc, dễ bảo trì	Khó hiểu với nhiều bảng và điều kiện phức tạp	Dễ hiểu hơn, cấu trúc rõ ràng, tách riêng JOIN
Hỗ trợ JOIN ngoài (OUTER)	Không hỗ trợ LEFT, RIGHT, FULL	Hỗ trợ đầy đủ JOIN (INNER, LEFT, RIGHT, FULL)
Tính tương thích	Cú pháp cũ, chuẩn SQL-89	Cú pháp hiện đại, chuẩn SQL-92 trở về sau
Ví dụ tương đương	FROM A, B WHERE A.id = B.id	FROM A INNER JOIN B ON A.id = B.id

- Cả hai cách đều cho kết quả giống nhau với điều kiện JOIN đơn giản (INNER JOIN).
- Tuy nhiên, JOIN là cách viết hiện đại, rõ ràng, dễ mở rộng với OUTER JOIN và SUBQUERY.
- Nên sử dụng JOIN trong thiết kế truy vấn chuyên nghiệp và dễ bảo trì.



10.3. Nhóm và tổng hợp dữ liệu trong SQL

- Mục đích:
 - Tổng hợp dữ liệu theo nhóm – thường dùng trong báo cáo, thống kê, hoặc phân tích dữ liệu
 - Kết hợp với hàm tổng hợp như COUNT, SUM, AVG, MIN, MAX
- Ý nghĩa:
 - Từ dữ liệu chi tiết → rút trích thông tin tổng hợp có ý nghĩa
 - Ví dụ: tổng lương theo phòng, số nhân viên mỗi phòng

Hàm tổng hợp trong SQL

Hàm tổng hợp	Ý nghĩa	Ví dụ sử dụng
COUNT()	Đếm số dòng	COUNT(*), COUNT(MANV)
SUM()	Tổng giá trị	SUM(LUONG)
AVG()	Trung bình cộng	AVG(LUONG)
MIN()	Giá trị nhỏ nhất	MIN(LUONG)
MAX()	Giá trị lớn nhất	MAX(LUONG)

Mệnh đề **GROUP BY**

- Mục đích:
 - Nhóm dữ liệu theo 1 hoặc nhiều cột, sau đó áp dụng hàm tổng hợp trên từng nhóm.
- Cú pháp:

```
SELECT <Cột_nhóm>, <Hàm_tổng_hợp>  
FROM <Bảng>  
GROUP BY <Cột_nhóm>;
```
- Ví dụ:

```
SELECT PHG, AVG(LUONG) AS LUONG_TB  
FROM NHANVIEN  
GROUP BY PHG;
```
- *Dữ liệu được nhóm theo PHG (phòng ban)*
- *Trả về lương trung bình của mỗi phòng*

Mệnh đề **HAVING**

- Mục đích:
 - Lọc kết quả sau khi đã nhóm dữ liệu bằng GROUP BY (*Lưu ý: khác với WHERE, dùng trước khi nhóm*).
SELECT <Cột_nhóm>, <Hàm_tổng_hợp>
FROM <Bảng>
GROUP BY <Cột_nhóm>
HAVING <Điều_kiện_trên_nhóm>;
- Ví dụ:
SELECT PHG, COUNT(*) AS SoNV
FROM NHANVIEN
GROUP BY PHG
HAVING COUNT(*) > 2;
- *Đếm số nhân viên từng phòng*
- *Chỉ hiện các phòng có trên 2 người*

So sánh WHERE vs HAVING

Tiêu chí	WHERE	HAVING
Áp dụng lúc nào	Trước khi nhóm (GROUP BY)	Sau khi nhóm (GROUP BY)
Dùng với hàm tổng hợp	✗ Không hỗ trợ	✓ Có thể dùng SUM, AVG,...
Ví dụ	WHERE PHG = 1	HAVING COUNT(*) > 2

- bc

Ví dụ 4 tổng hợp có đủ cả GROUP BY + HAVING

- Truy xuất những phòng có nhân viên lương cao nhất ≥ 15000

```
SELECT PHG, MAX(LUONG) AS MAX_LUONG  
FROM NHANVIEN  
GROUP BY PHG  
HAVING MAX(LUONG) >= 15000;
```

- GROUP BY là bắt buộc khi dùng hàm tổng hợp trên từng nhóm
- HAVING được dùng để lọc nhóm theo điều kiện tổng hợp
- Kết hợp GROUP BY, HAVING và hàm tổng hợp cho phép truy vấn phân tích linh hoạt

10.4. Truy vấn lồng nhau (Subquery)

Tổng quan về Subquery



- Mục đích:
 - Cho phép thực hiện truy vấn bên trong một truy vấn khác
 - Giúp xử lý các yêu cầu phức tạp như:
 - ✓ So sánh với dữ liệu tổng hợp
 - ✓ Truy vấn theo điều kiện phụ thuộc vào dữ liệu khác
- Ý nghĩa:
 - Subquery thường trả về 1 giá trị, 1 tập giá trị, hoặc 1 bảng tạm

Ví dụ 5

```
SELECT MANV, TENV, TENNV
```

```
FROM NHANVIEN, PHONGBAN
```

```
WHERE TENPHG = N'Nguyễn Phú'
```

```
from PHONGBAN pb
```

```
where pb.TENPHG = N'Nguyễn cứu')
```

Câu truy vấn cha
(Outer query)

```
SELECT <danh sách các cột>
```

```
FROM <danh sách các bảng>
```

```
WHERE <so sánh tập hợp> (
```

```
SELECT <danh sách các cột>
```

```
FROM <danh sách các bảng>
```

```
WHERE <điều kiện>)
```

Câu truy vấn con
(Subquery)

Truy vấn lồng

- Các câu lệnh SELECT có thể lồng nhau ở nhiều mức
- Các câu truy vấn con trong cùng một mệnh đề WHERE được kết hợp bằng phép nối logic
- Câu truy vấn con thường trả về một tập các giá trị
- Mệnh đề WHERE của câu truy vấn cha
 - <biểu thức> <so sánh tập hợp> <truy vấn con>
 - So sánh tập hợp thường đi cùng với một số toán tử
 - ✓ IN, NOT IN
 - ✓ ALL
 - ✓ ANY hoặc SOME
 - Kiểm tra sự tồn tại
 - ✓ EXISTS - NOT EXISTS

- Có 2 loại truy vấn lồng
 - Lồng phân cấp
 - ✓ Mệnh đề WHERE của truy vấn con **không tham chiếu** đến thuộc tính của các quan hệ trong mệnh đề FROM ở truy vấn cha
 - ✓ Khi thực hiện, câu truy vấn con sẽ được thực hiện trước
 - Lồng tương quan
 - ✓ Mệnh đề WHERE của truy vấn con **tham chiếu ít nhất** một thuộc tính của các quan hệ trong mệnh đề FROM ở truy vấn cha
 - ✓ Khi thực hiện, câu truy vấn con sẽ được thực hiện nhiều lần, mỗi lần tương ứng với một bộ của truy vấn cha

Ví dụ 6 - Lồng phân cấp

```
SELECT MANV, TENNV  
FROM NHANVIEN, DIADIEM_PHG  
WHERE DIADIEM='TP HCM' AND PHG=MAPHG
```

```
SELECT MANV, TENNV  
FROM NHANVIEN  
WHERE PHG IN (1, 5) (
```

```
SELECT MAPHG  
FROM DIADIEM_PHG  
WHERE DIADIEM='TP HCM' )
```

Ví dụ 7

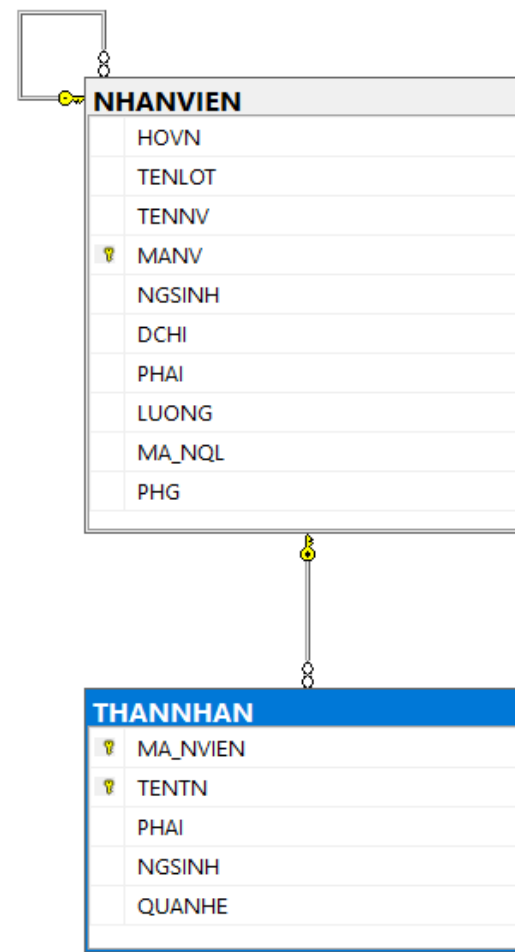
```
SELECT DISTINCT TENDA
  SELECT SODA
FROM DEAN
  FROM NHANVIEN NV, PHANCONG PC
WHERE MADA IN (
  WHERE NV.MANV=PC.MA_NVN AND NV.HONV='Nguyen'
  SELECT SODA
  UNION
  FROM NHANVIEN, PHANCONG
  SELECT MADA
  WHERE MANV=MA_NVN AND HONV='Nguyen' )
FROM NHANVIEN NV, PHONGBAN PB, DEAN DA
OR MADA IN (
  WHERE NV.MANV=PB.TRPHG AND PB.MAPHG=DA.PHONG
  SELECT MADA
  AND NV.HONV='Nguyen'
  FROM NHANVIEN, PHONGBAN, DEAN
  WHERE MANV=TRPHG AND MAPHG=PHONG
  AND HONV='Nguyen' )
```

Ví dụ 8

- Tìm những nhân viên không có thân nhân nào

```
SELECT *  
FROM NHANVIEN  
WHERE MANV NOT IN (  
    SELECT MA_NVIE  
    FROM THANNHAN )
```

```
SELECT *  
FROM NHANVIEN  
WHERE MANV <> ALL (  
    SELECT MA_NVIE  
    FROM THANNHAN )
```



Ví dụ 9



- Tìm những nhân viên có lương lớn hơn lương của ít nhất một nhân viên phòng 4

```
SELECT *  
FROM NHANVIEN  
WHERE LUONG > ANY (  
    SELECT LUONG  
    FROM NHANVIEN  
    WHERE PHG=4 )
```

```
SELECT DISTINCT NV1.*  
FROM NHANVIEN NV1, NHANVIEN NV2  
WHERE NV1.LUONG > NV2.LUONG AND NV2.PHG=4
```

Ví dụ 10

- Tìm những nhân viên có lương lớn hơn lương của tất cả nhân viên phòng 4

```
SELECT *  
FROM NHANVIEN  
WHERE LUONG > ALL (  
    SELECT LUONG  
    FROM NHANVIEN  
    WHERE PHG=4 )
```


Ví dụ 11

- Tìm những trưởng phòng có tối thiểu một thân nhân

```
SELECT *  
FROM NHANVIEN  
WHERE MANV IN (SELECT MA_NVIEN FROM THANNHAN)  
AND MANV IN (SELECT TRPHG FROM PHONGBAN)
```

Ví dụ - Lòng tương quan

```
SELECT MANV, TENNV  
FROM NHANVIEN, PHONGBAN  
WHERE TENPHG='Nghien cuu' AND PHG=MAPHG
```

```
SELECT MANV, TENNV  
FROM NHANVIEN  
WHERE EXISTS (  
    SELECT *  
    FROM PHONGBAN  
    WHERE TENPHG='Nghien cuu' AND PHG=MAPHG )
```

Ví dụ 12

- Tìm nhân viên có người thân cùng tên và cùng giới tính

```
SELECT *  
FROM NHANVIEN NV  
WHERE EXISTS (  
    SELECT *  
    FROM THANNHAN TN  
    WHERE NV.MANV=TN.MA_NVIENT  
    AND NV.TENNV=TN.TENTN  
    AND NV.PHAI=TN.PHAI )
```

Ví dụ 13

- Tìm những nhân viên không có thân nhân nào

```
SELECT *  
FROM NHANVIEN  
WHERE NOT EXISTS (  
    SELECT *  
    FROM THANNHAN  
    WHERE MANV=MA_NVIENT)
```

Ví dụ 14



- Tìm những nhân viên có lương lớn hơn lương của ít nhất một nhân viên phòng 4

```
SELECT *  
FROM NHANVIEN NV1  
WHERE EXISTS (  
    SELECT *  
    FROM NHANVIEN NV2  
    WHERE NV2PHG=4  
    AND NV1.LUONG>NV2.LUONG)
```

Ví dụ 15

- Tìm những trưởng phòng có tối thiểu một thân nhân

```
SELECT *  
FROM NHANVIEN  
WHERE EXISTS (  
    SELECT *  
    FROM THANNHAN  
    WHERE MANV=MA_NVIEN )  
AND EXISTS (  
    SELECT *  
    FROM PHONGBAN  
    WHERE MANV=TRPHG )
```


Nhận xét IN và EXISTS

- IN

- <tên cột> IN <câu truy vấn con>

- Thuộc tính ở mệnh đề SELECT của truy vấn con phải có cùng kiểu dữ liệu với thuộc tính ở mệnh đề WHERE của truy vấn cha

- EXISTS

- Không cần có thuộc tính, hằng số hay biểu thức nào khác đứng trước

- Không nhất thiết liệt kê tên thuộc tính ở mệnh đề SELECT của truy vấn con

- Những câu truy vấn có = ANY hay IN đều có thể chuyển thành câu truy vấn có

EXISTS

Phép chia trong SQL

R	A	B	C	D	E
	α	a	α	a	1
	α	a	γ	a	1
	α	a	γ	b	1
	β	a	γ	a	1
	β	a	γ	b	3
	γ	a	γ	a	1
	γ	a	γ	b	1
	γ	a	β	b	1

S	D	E
b_i	a	1
	b	1

$R \div S$	A	B	C
a_i	α	a	γ
	γ	a	γ

- $R \div S$ là tập các giá trị a_i trong R sao cho không có giá trị b_i nào trong S làm cho bộ (a_i, b_i) không tồn tại trong R

Phép chia trong SQL (tt)

- Sử dụng NOT EXISTS để biểu diễn

```
SELECT R1.A, R1.B, R1.C
```

```
FROM R R1
```

```
WHERE NOT EXISTS (
```

```
    SELECT *
```

```
    FROM S
```

```
    WHERE NOT EXISTS (
```

```
        SELECT *
```

```
        FROM R R2
```

```
        WHERE R2.D=S.D AND R2.E=S.E
```

```
        AND R1.A=R2.A AND R1.B=R2.B AND R1.C=R2.C ))
```

Ví dụ 16

- Tìm tên các nhân viên được phân công làm tất cả các đề án
 - Tìm tên các **nhân viên** mà không có **đề án** nào là không được **phân công** làm
 - Tập bị chia: **PHANCONG**(MA_NVIEN, MADA)
 - Tập chia: **DEAN**(MADA)
 - Tập kết quả: KQ(MA_NVIEN)
 - Kết KQ với **NHANVIEN** để lấy ra TENNV

Ví dụ 16 (tt)

```
SELECT NV.MANV, NV.TENNV
FROM NHANVIEN NV, PHANCONG PC1
WHERE NV.MANV=PC1.MA_NVIENT
AND NOT EXISTS (
    SELECT *
    FROM DEAN DA
    WHERE NOT EXISTS (
        SELECT *
        FROM PHANCONG PC2
        WHERE PC2.MADA=DA.MADA
        AND PC1.MA_NVIENT=PC2.MA_NVIENT ))
```

Ví dụ 16 (tt)



Cách 2

```
SELECT NV.MANV, NV.TENNV
FROM NHANVIEN NV
WHERE NOT EXISTS (
    SELECT *
    FROM DEAN DA
    WHERE NOT EXISTS (
        SELECT *
        FROM PHANCONG PC
        WHERE PC.MADA=DA.MADA
        AND NV.MANV = PC.MA_NVIENT
```

```
))
```

So sánh Subquery và JOIN

Tiêu chí	Subquery	JOIN
Cách viết	Lồng truy vấn bên trong	Ghép bảng theo khóa
Hiệu suất	Có thể kém nếu lồng nhiều cấp	JOIN nhanh hơn khi tối ưu
Dùng khi	So sánh dữ liệu tạm/tổng hợp	Lấy thông tin liên bảng
Ví dụ phù hợp	WHERE có tồn tại/tổng hợp	Liên kết thông tin nhân viên – phòng ban

- Subquery giúp xử lý các truy vấn phức tạp, linh hoạt
- Có thể dùng ở các vị trí: WHERE, FROM, SELECT
- Kết hợp với các toán tử: IN, EXISTS, ANY, ALL
- Tuy nhiên, với truy vấn lớn và đa bảng → nên cân nhắc JOIN để tối ưu hiệu suất

a

- bc

